

Albertian Institute of Science & Technology

(Approved by AICTE, Affiliated to State Board of Technical Education)

Lab Manual for

Python Programming Lab – 3347

Semester – III (AI & ML)

Prepared By,

Kalyani M Biju

Lecturer, Department of Artificial Intelligence & Machine Learning

INDEX

Exp No.	Name of Experiment	Date	Page No.	Faculty Signature
1	Area Of Triangle			
2	Largest Among Three Numbers			
3	Palindrome Numbers			
4	Palindrome String			
5	Fibonacci Series			
6	Vowels And Consonants			
7	Sorting			
8	Maximum And Minimum Element Of A Tuple			
9	List Of Frequencies			
10	Frequency Of Characters Of A File			
11	Exception Handling			
12	Student Class			
13	Inheritance			
14	Numpy			
15	Panda Module			
16	Matplotlib			

EXPERIMENT NO: 1 **Date:** _____

AREA OF TRIANGLE

AIM

Write a program to Find the area of a right angled triangle

OBJECTIVES

- To understand fundamental user input operations and type casting in Python.
- To execute arithmetic multiplication and division calculations for geometry.
- To format and display computed numerical results to the console output.

ALGORITHM

Step 1 :- Initialize the execution environment to accept input values for base and height of the right angled triangle.

Step 2 :- Prompt the user to enter the base length of the triangle and convert it to an integer.

Step 3 :- Prompt the user to enter the height of the triangle and convert it to an integer.

Step 4 :- Multiply the base and height values together and store the result in variable c.

Step 5 :- Divide the product c by two to calculate the area of the right angled triangle.

Step 6 :- Output the calculated area value along with descriptive text to the standard console display window.

Step 7 :- Terminate the execution of the Python program successfully and release system memory resources.

PROGRAM

```
b = int(input("Enter the base"))
h = int(input("Enter the height"))
c = b * h
area = c / 2
print("Area of right angle triangle is:", area, "sqcm")
```

SAMPLE OUTPUT

Terminal Output

```
Enter the base:  5
Enter the height:  10
Area of right angle triangle is:  25.0 sqcm
```

RESULT

The program to calculate the area of a right-angled triangle was successfully implemented and verified.

EXPERIMENT NO: 2 **Date:** _____

LARGEST AMONG THREE NUMBERS

AIM

Write a program to Find the greatest among three numbers

OBJECTIVES

- To implement multi-way branching using conditional control structures.
- To utilize relational and logical AND operators for comparison.
- To determine maximum numerical value among user inputs.

ALGORITHM

Step 1 :- Begin execution of the program to compare three user input numbers and find the largest value.

Step 2 :- Prompt the user to enter the first number and convert the input string to an integer.

Step 3 :- Prompt the user to enter the second number and convert the input string to an integer.

Step 4 :- Prompt the user to enter the third number and convert the input string to an integer.

Step 5 :- Check if the first number is strictly greater than both second and third input numbers.

Step 6 :- Check if the second number is strictly greater than the third number if first check fails.

Step 7 :- Consider the third number as largest if both previous conditional checks evaluate to false.

Step 8 :- Print the identified largest number to the terminal display with an appropriate descriptive text message.

Step 9 :- Conclude the execution of the number comparison program and return control to the operating system.

PROGRAM

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))
if a > b and a > c:
    print("largest number is ", a)
elif b > c:
    print("the largest number is ", b)
else:
    print("the largest number is ", c)
```

SAMPLE OUTPUT

Terminal Output

```
Enter first number:  10
Enter second number:  5
Enter third number:  2
largest number is 10
```

RESULT

The program to find the largest among three numbers was successfully executed and verified.

EXPERIMENT NO: 3 **Date:** _____

PALINDROME NUMBERS

AIM

Write a program to check whether a given number is palindrome or not

OBJECTIVES

- To implement indefinite iteration using while loop structure.
- To perform digit extraction using modulus and integer floor division.
- To reverse integer values and verify numeric symmetry.

ALGORITHM

Step 1 :- Begin the program to check whether a given user input integer number is a palindrome or not.

Step 2 :- Prompt the user to enter a number and store it in variable n and temporary variable temp.

Step 3 :- Set the initial value of the reversed number variable rev to zero before starting loop.

Step 4 :- Execute a while loop as long as the temporary variable temp is strictly greater than zero.

Step 5 :- Calculate the remainder digit by taking modulus of temporary variable temp by base value ten.

Step 6 :- Multiply rev by ten and add extracted remainder digit to accumulate reversed numerical value.

Step 7 :- Perform integer floor division on variable temp by ten to remove the last processed digit.

Step 8 :- Compare the reversed number rev with original number n for exact numerical equality match.

Step 9 :- Print palindrome if numbers are equal, otherwise print not palindrome to console display window.

Step 10 :- Terminate the execution of the palindrome check Python script and clear runtime variables from memory.

PROGRAM

```
n = int(input("enter a number"))
rev = 0
temp = n
while temp > 0:
    rem = temp % 10
    rev = rev * 10 + rem
    temp = temp // 10

if rev == n:
    print("palindrome")
else:
    print("not palindrome")
```

SAMPLE OUTPUT

Terminal Output

```
enter a number:  121
palindrome
```

RESULT

The palindrome number checking program was executed and verified for various input cases.

EXPERIMENT NO: 4 **Date:** _____

PALINDROME STRING

AIM

Write a program to check whether a given string is palindrome or not

OBJECTIVES

- To understand string indexing and slicing operations in Python.
- To reverse a text string starting from the last character to the first character.
- To compare the original string with the reversed string for palindrome validation.

ALGORITHM

Step 1 :- Initialize the execution environment to accept string input and evaluate string palindrome symmetry.

Step 2 :- Prompt the user to enter a text string and store the input in variable s.

Step 3 :- Reverse the string by slicing from back to front and store in variable rev.

Step 4 :- Compare the original string s with the reversed string rev for exact string equality.

Step 5 :- Display palindrome message if strings match, otherwise print not a palindrome message to console.

Step 6 :- Terminate the execution of the Python program successfully and free runtime resources back to system.

PROGRAM

```
s = input("Enter a string: ")
rev = s[::-1]
if s == rev:
    print("The string is a palindrome")
else:
    print("The string is not a palindrome")
```

SAMPLE OUTPUT

Terminal Output

```
Enter a string:  madam
The string is a palindrome
```

RESULT

The program to check whether a given string is a palindrome was successfully implemented and output verified.

EXPERIMENT NO: 5 **Date:** _____

FIBONACCI SERIES

AIM

Write a program to print Fibonacci series

OBJECTIVES

- To understand sequence generation and iterative loop bounds.
- To manage variable swapping and accumulator updates during iterations.
- To display sequence terms up to user specified limit.

ALGORITHM

Step 1 :- Initialize the execution environment to compute and display a specified number of Fibonacci series terms.

Step 2 :- Prompt user to enter total number of terms and store integer value in variable n.

Step 3 :- Set first term variable a to zero and second term variable b to value one.

Step 4 :- Print the first two seed numbers a and b on separate lines to console.

Step 5 :- Loop through index range starting from two up to specified total term count n.

Step 6 :- Add current values of variables a and b to compute the next term c.

Step 7 :- Print the newly calculated Fibonacci term c to standard terminal console display screen.

Step 8 :- Update variable a with value of b, and update variable b with new value c.

Step 9 :- Terminate the execution of Fibonacci series generation program and free allocated system memory resources.

PROGRAM

```
n = int(input("enter your number of terms"))
a = 0
b = 1
print(a)
print(b)
for i in range(2, n):
    c = a + b
    print(c)
    a = b
    b = c
```

SAMPLE OUTPUT

Terminal Output

```
enter your number of terms:  7
0
1
1
2
3
5
8
```

RESULT

The program to generate the Fibonacci series was implemented and output verified successfully.

EXPERIMENT NO: 6 **Date:** _____

VOWELS AND CONSONANTS

AIM

Write a program to count number of vowels and consonants in a string

OBJECTIVES

- To iterate over character elements of a text string.
- To perform string inspection using logical OR conditions and isalpha().
- To count distinct character classes in textual input.

ALGORITHM

Step 1 :- Begin the execution of string character parsing program to count total vowels and consonants.

Step 2 :- Prompt user to enter a string and store string value in variable s.

Step 3 :- Find total character count of string s using len function and store in l.

Step 4 :- Set vowel count variable v to zero and consonant count variable c to zero.

Step 5 :- Loop through character index range from index zero up to string length limit l.

Step 6 :- Check if character at index i matches any lowercase or uppercase vowel letter.

Step 7 :- Add one to vowel counter v if character matches vowel comparison condition check.

Step 8 :- Check if character is alphabetic letter using isalpha and increment consonant counter c.

Step 9 :- Print total count of vowels and consonants to standard console terminal output window.

Step 10 :- Conclude execution of string parsing script and release string buffer memory back to system.

PROGRAM

```
s = str(input("enter a string"))
l = len(s)
c = 0
v = 0
for i in range(l):
    if s[i] == 'a' or s[i] == 'e' or s[i] == 'i' or s[i] == 'o' or s[i] == 'u' or \
        s[i] == 'A' or s[i] == 'E' or s[i] == 'I' or s[i] == 'O' or s[i] == 'U':
         v = v + 1
    elif s[i].isalpha():
         c = c + 1
print("Number of vowels is ", v)
print("Number of consonants is ", c)
```

SAMPLE OUTPUT

Terminal Output

```
enter a string:  Hello
Number of vowels is 2
Number of consonants is 3
```

RESULT

The vowel and consonant counting program was successfully executed and output verified.

EXPERIMENT NO: 7 **Date:** _____

SORTING

AIM

Write a program to Sort the elements of a list using function

OBJECTIVES

- To define modular functions accepting list parameters in Python.
- To implement bubble sort algorithm using nested loops and element swapping.
- To return and print ordered data structures.

ALGORITHM

Step 1 :- Initialize the bubble sort program to rearrange list elements in non decreasing numerical order.

Step 2 :- Declare bubble sort function taking list argument arr to perform element sorting operations.

Step 3 :- Calculate total element count of input list arr and store integer length in n.

Step 4 :- Execute outer loop for variable i ranging from index zero to length n minus one.

Step 5 :- Execute inner loop for variable j ranging from index zero to n minus i minus one.

Step 6 :- Check if element at index j is strictly greater than adjacent element at j plus one.

Step 7 :- Swap adjacent elements in place if left element is larger than right element.

Step 8 :- Return ordered list arr from bubble sort function after completing all comparison passes.

Step 9 :- Call sorting function with sample list and print sorted result to terminal window.

Step 10 :- Conclude execution of sorting algorithm program and free list memory allocation from system.

PROGRAM

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

sample_list = [64, 34, 25, 12, 22, 11, 90]
sorted_list = bubble_sort(sample_list)
print(f"Sorted list in non-decreasing order: {sorted_list}")
```

SAMPLE OUTPUT

Terminal Output

```
Sorted list in non-decreasing order:  [11, 12, 22, 25, 34, 64, 90]
```

RESULT

The list sorting function was successfully implemented using bubble sort and output verified.

EXPERIMENT NO: 8 **Date:** _____

MAXIMUM AND MINIMUM ELEMENT OF A TUPLE

AIM

Write a program to find Maximum and minimum elements of a tuple using function

OBJECTIVES

- To explore immutable sequence types (tuples) in Python.
- To traverse tuple elements and perform conditional minimum/maximum evaluation.
- To return tuple pairs containing multiple values from functions.

ALGORITHM

Step 1 :- Initialize program execution to find maximum and minimum elements of a tuple using function.

Step 2 :- Declare function find max min accepting tuple parameter tup for processing element extremes.

Step 3 :- Return None for both maximum and minimum if input tuple tup is empty.

Step 4 :- Set initial variables max element and min element to first element of tup.

Step 5 :- Iterate through each numeric element num present inside input tuple tup in order.

Step 6 :- Compare num with max element and update max element if num is strictly larger.

Step 7 :- Compare num with min element and update min element if num is strictly smaller.

Step 8 :- Return tuple containing calculated max element and min element from custom function.

Step 9 :- Pass sample tuple to function and print maximum and minimum values to terminal screen.

Step 10 :- Conclude tuple analysis program execution successfully and clear tuple structures from memory.

PROGRAM

```
def find_max_min(tup):
    if not tup:
        return None, None
    max_element = tup[0]
    min_element = tup[0]

    for num in tup:
        if num > max_element:
            max_element = num
        if num < min_element:
            min_element = num
    return max_element, min_element

```

```
sample_tuple = (3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5)
max_element, min_element = find_max_min(sample_tuple)
print(f"The maximum element in the tuple is: {max_element}")
print(f"The minimum element in the tuple is: {min_element}")
```

SAMPLE OUTPUT

Terminal Output

```
The maximum element in the tuple is:  9
The minimum element in the tuple is:  1
```

RESULT

The function to find maximum and minimum elements of a tuple was executed and verified.

EXPERIMENT NO: 9 **Date:** _____

LIST OF FREQUENCIES

AIM

Write a program to create list of frequencies in non-increasing order using function

OBJECTIVES

- To build frequency maps using Python dictionaries.
- To extract dictionary values into lists and implement custom sorting logic.
- To arrange frequency counts in descending numerical order.

ALGORITHM

Step 1 :- Initialize frequency distribution analyzer program to count list occurrences and sort in descending order.

Step 2 :- Declare function frequency list accepting array parameter arr for calculating item occurrence counts.

Step 3 :- Return empty list immediately if input list arr contains no numerical elements.

Step 4 :- Iterate items in list arr, updating element counts inside dictionary frequency dict.

Step 5 :- Convert dictionary count values into linear list frequencies for sorting processing.

Step 6 :- Run outer loop over frequency list indices to begin descending bubble sort algorithm.

Step 7 :- Run inner loop comparing current frequency against subsequent elements in frequencies list.

Step 8 :- Swap elements if preceding frequency count is smaller than following frequency count.

Step 9 :- Return sorted frequency list from function and print result to standard output screen.

Step 10 :- Conclude execution of frequency counting script and release dictionary memory back to system.

PROGRAM

```
def frequency_list(arr):
    if not arr:
        return []
    frequency_dict = {}

    for item in arr:
        if item in frequency_dict:
            frequency_dict[item] += 1
        else:
            frequency_dict[item] = 1
    frequencies = list(frequency_dict.values())

    for i in range(len(frequencies)):
```

```
        for j in range(i + 1, len(frequencies)):
            if frequencies[i] < frequencies[j]:
                frequencies[i], frequencies[j] = frequencies[j], frequencies
    [i]

    return frequencies

sample_list = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
frequencies = frequency_list(sample_list)
print(f"List of frequencies in non-increasing order: {frequencies}")
```

SAMPLE OUTPUT

Terminal Output

```
List of frequencies in non-increasing order:  [3, 2, 2, 1, 1, 1, 1]
```

RESULT

The frequency list computation program in non-increasing order was tested and verified.

EXPERIMENT NO: 10 **Date:** _____

FREQUENCY OF CHARACTERS OF A FILE

AIM

Write a program to find Frequency of characters in a given file

OBJECTIVES

- To manage file I/O operations using Python context managers (with open).
- To handle file stream exceptions like FileNotFoundError.
- To count character occurrences across file text content.

ALGORITHM

Step 1 :- Initialize character frequency counting program to process and count character occurrences in a file.

Step 2 :- Create function character frequency taking filename argument to read file and count characters.

Step 3 :- Open file in read mode safely using open context manager and store content in text.

Step 4 :- Iterate through each character char in text, updating counts in dictionary frequency dict.

Step 5 :- Intercept FileNotFoundError exception using try except block if target file does not exist.

Step 6 :- Open file.txt in write mode, write string hello world, and close file handle.

Step 7 :- Invoke character frequency function with filename and print calculated frequency dictionary to console.

Step 8 :- Finish file operation execution context and release file stream resources back to operating system.

PROGRAM

```
def character_frequency(filename):
    try:
        with open(filename, 'r') as file:
            text = file.read()
        frequency_dict = {}
        for char in text:
            if char in frequency_dict:
                frequency_dict[char] += 1
            else:
                frequency_dict[char] = 1
        return frequency_dict
    except FileNotFoundError:
        print(f"The file {filename} does not exist.")
        return {}

```

```
filename = 'file.txt'
f = open(filename, 'w')
f.write("hello world")
f.close()

frequencies = character_frequency(filename)
print(f"Frequencies of characters in the file: {frequencies}")
```

SAMPLE OUTPUT

Terminal Output

```
Frequencies of characters in the file: 'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1,
'w': 1, 'r': 1, 'd': 1
```

RESULT

The file character frequency analyzer was implemented and output verified successfully.

EXPERIMENT NO: 11 **Date:** _____

EXCEPTION HANDLING

AIM

Write a program to handle exception in the case of voters age validity If age ¡ 0 raise an exception

OBJECTIVES

- To raise custom exceptions (ValueError) based on business logic.
- To wrap error prone statements inside try-except blocks.
- To intercept specific exception classes (ValueError, TypeError).

ALGORITHM

Step 1 :- Begin voter eligibility checking program with custom exception handling for negative age values.

Step 2 :- Declare function check voter age accepting numerical integer age parameter for validation evaluation.

Step 3 :- Raise ValueError exception with error message if input age is strictly less than zero.

Step 4 :- Print not eligible if age is under eighteen, or eligible if age is eighteen or above.

Step 5 :- Prompt user for integer age input inside try block and call check voter age function.

Step 6 :- Catch raised ValueError exception and print descriptive error message to terminal console.

Step 7 :- Catch TypeError exception if user provides invalid non integer input for age variable.

Step 8 :- Conclude exception handling program safely without crashing application runtime environment on invalid user inputs.

PROGRAM

```
def check_voter_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative.")
    elif age < 18:
        print("You are not eligible to vote.")
    else:
        print("You are eligible to vote.")

try:
    age = int(input("Enter your age: "))
    check_voter_age(age)
except ValueError as e:
    print(f"Error: {e}")
except TypeError:
    print("Please enter a valid integer for age.")
```

SAMPLE OUTPUT

Terminal Output

```
Enter your age:  -5
Error:  Age cannot be negative.
```

RESULT

The voter age exception handling program was executed and exception handling verified.

EXPERIMENT NO: 12 **Date:** _____

STUDENT CLASS

AIM

Write a program to create a student class and create following methods: setMarks() - to update marks, getMarks() - to get individual marks, avgMarks() - to find average of marks, showResults() - to show results of each student

OBJECTIVES

- To understand Object-Oriented Programming (OOP) concepts in Python.
- To initialize instance attributes using constructors ($_{init}$).
- To define instance methods for data manipulation, query, and formatted summary display.

ALGORITHM

Step 1 :- Initialize student management application using object oriented programming concepts to model student scores.

Step 2 :- Define Student class with constructor initializing name attribute and empty marks dictionary.

Step 3 :- Define setMarks method taking subject and mark parameters to update student marks dictionary.

Step 4 :- Define getMarks method to return individual subject score or default missing message string.

Step 5 :- Define avgMarks method calculating average by dividing sum of marks by total subjects.

Step 6 :- Define showResults method printing subject scores breakdown and computed average mark summary.

Step 7 :- Create Student object student1, set marks for subjects, and call class methods to display results.

Step 8 :- Conclude execution of object oriented student class program and destroy object instance from memory.

PROGRAM

```
class Student:
    def __init__(self, name):
        self.name = name
        self.marks = {}

    def setMarks(self, subject, mark):
        self.marks[subject] = mark

    def getMarks(self, subject):
        return self.marks.get(subject, "No marks available for this subject
    .")
```

```
11
12      def avgMarks(self):
13          if not self.marks:
14              return "No marks available."
15          total_marks = sum(self.marks.values())
16          average_marks = total_marks / len(self.marks)
17          return average_marks
18
19      def showResults(self):
20          if not self.marks:
21              print(f"No marks available for {self.name}.")
22              return
23          print(f"Results for {self.name}:")
24          for subject, mark in self.marks.items():
25              print(f"{subject}: {mark}")
26          average_marks = self.avgMarks()
27          print(f"Average Marks: {average_marks}")
28
29  student1 = Student("John Doe")
30  student1.setMarks("Math", 85)
31  student1.setMarks("Science", 90)
32  student1.setMarks("English", 88)
33
34  print(student1.getMarks("Math"))
35  print(student1.getMarks("History"))
36  print(student1.avgMarks())
37  student1.showResults()
```

SAMPLE OUTPUT

Terminal Output

```
85
No marks available for this subject.
87.66666666666667
Results for John Doe:
Math:  85
Science:  90
English:  88
Average Marks:  87.66666666666667
```

RESULT

The Student class implementation with method encapsulation was verified successfully.

EXPERIMENT NO: 13 **Date:** _____

INHERITANCE

AIM

Write a program to create a vehicle class with variables name, speed, mileage and methods setCapacity() and getFare(), create a class bus that inherits from the Vehicle class.

OBJECTIVES

- To implement OOP inheritance between superclasses and subclasses.
- To leverage superclass methods using super().
- To demonstrate method overriding and specialized behavior in derived classes.

ALGORITHM

Step 1 :- Initialize inheritance program to model base Vehicle class and derived Bus child class behaviors.

Step 2 :- Define base Vehicle class with constructor setting name, speed, mileage, and initial capacity.

Step 3 :- Add setCapacity method to store passenger limit and getFare method calculating base fare.

Step 4 :- Create child Bus class inheriting from Vehicle, overriding getFare method with additional charge.

Step 5 :- Create Vehicle object vehicle1, set capacity to five, and display vehicle properties and fare.

Step 6 :- Create Bus object bus1, set capacity to fifty, and display inherited properties and calculated fare.

Step 7 :- Conclude object oriented inheritance program execution and release class instances back to memory heap.

PROGRAM

```
class Vehicle:
    def __init__(self, name, speed, mileage):
        self.name = name
        self.speed = speed
        self.mileage = mileage
        self.capacity = 0

    def setCapacity(self, capacity):
        self.capacity = capacity

    def getFare(self):
        return self.capacity * 100

class Bus(Vehicle):
    def getFare(self):
        base_fare = super().getFare()
        return base_fare * 1.1
```

```
18
19 vehicle1 = Vehicle("Car", 150, 15)
20 vehicle1.setCapacity(5)
21 print(f"Vehicle Name: {vehicle1.name}")
22 print(f"Speed: {vehicle1.speed} km/h")
23 print(f"Mileage: {vehicle1.mileage} km/l")
24 print(f"Capacity: {vehicle1.capacity}")
25 print(f"Fare: {vehicle1.getFare()}")
26
27 bus1 = Bus("School Bus", 80, 10)
28 bus1.setCapacity(50)
29 print(f"\nBus Name: {bus1.name}")
30 print(f"Speed: {bus1.speed} km/h")
31 print(f"Mileage: {bus1.mileage} km/l")
32 print(f"Capacity: {bus1.capacity}")
33 print(f"Fare: {bus1.getFare()}")
```

SAMPLE OUTPUT

Terminal Output

```
Vehicle Name:  Car
Speed:  150 km/h
Mileage:  15 km/l
Capacity:  5
Fare:  500

Bus Name:  School Bus
Speed:  80 km/h
Mileage:  10 km/l
Capacity:  50
Fare:  5500.0
```

RESULT

The OOP inheritance hierarchy between Vehicle and Bus classes was successfully demonstrated.

EXPERIMENT NO: 14 **Date:** _____

NUMPY

AIM

Write a program to create a matrix using numpy and do the following operations: Perform matrix addition, subtraction and multiplication, Find transpose of matrix, Calculates the inverse of matrix, Calculates the determinant of matrix, Transforms a matrix into 1d array (flatten)

OBJECTIVES

- To utilize NumPy library for numerical array processing.
- To perform fundamental matrix operations (addition, subtraction, dot product).
- To compute linear algebra functions (transpose, inverse, determinant, flattening).

ALGORITHM

Step 1 :- Initialize matrix computation module using NumPy library for matrix arithmetic and linear algebra operations.

Step 2 :- Import numpy module under standard alias np to access array creation and matrix functions.

Step 3 :- Create two 2D arrays matrix1 and matrix2 with 2x2 integer values for calculations.

Step 4 :- Perform element wise addition, subtraction, and dot product multiplication using NumPy built in functions.

Step 5 :- Compute matrix transpose using np.transpose function to swap rows and columns of matrix1.

Step 6 :- Compute matrix inverse using np.linalg.inv and matrix determinant using np.linalg.det on matrix1.

Step 7 :- Transform 2D array matrix1 into 1D array flattened using matrix flatten method call.

Step 8 :- Print original matrices and calculated addition, subtraction, multiplication, transpose, inverse, determinant, and flattened outputs.

Step 9 :- Conclude NumPy matrix operations program execution and free array memory allocations from environment heap.

PROGRAM

```
import numpy as np

# Create matrices
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])

# Perform matrix addition
addition = np.add(matrix1, matrix2)

```

```
# Perform matrix subtraction
subtraction = np.subtract(matrix1, matrix2)

# Perform matrix multiplication
multiplication = np.dot(matrix1, matrix2)

# Find transpose of matrix1
transpose = np.transpose(matrix1)

# Calculate the inverse of matrix1
inverse = np.linalg.inv(matrix1)

# Calculate the determinant of matrix1
determinant = np.linalg.det(matrix1)

# Transform matrix1 into a 1D array (flatten)
flattened = matrix1.flatten()

# Display results
print("Matrix 1:\n", matrix1)
print("\nMatrix 2:\n", matrix2)
print("\nMatrix Addition:\n", addition)
print("\nMatrix Subtraction:\n", subtraction)
print("\nMatrix Multiplication:\n", multiplication)
print("\nTranspose of Matrix 1:\n", transpose)
print("\nInverse of Matrix 1:\n", inverse)
print("\nDeterminant of Matrix 1:\n", determinant)
print("\nFlattened Matrix 1:\n", flattened)
```

SAMPLE OUTPUT

Terminal Output

```
Matrix 1:
[1 2

4
]


Matrix 2:
[5 6

8

]


Matrix Addition:
[ 6 8

12

]


Matrix Subtraction:
[-4 -4

-4
]


Matrix Multiplication:
[19 22

50




]


Transpose of Matrix 1:
[1 3

4
]


Inverse of Matrix 1:
[-2.  1.

-0.5
]


Determinant of Matrix 1:
-2.0


Flattened Matrix 1:
2 3 4
```

RESULT

The matrix operations program using NumPy library was executed and results verified.

EXPERIMENT NO: 15 **Date:** _____

PANDA MODULE

AIM

Write a python program to read a .csv file using panda module and print the first and last five lines of a file

OBJECTIVES

- To leverage Pandas library for structured data manipulation.
- To load CSV datasets into DataFrames using $read_csv$.
- To inspect head and tail slices of dataset files.

ALGORITHM

Step 1 :- Initialize data processing program using Pandas module to read CSV file records and preview data lines.

Step 2 :- Import pandas library under alias pd to utilize DataFrame loading and inspection utility functions.

Step 3 :- Read external CSV dataset file using pd.$read_csvfunctionandstoredatainDataFramedf$.

Step 4 :- Call head method on DataFrame df to retrieve the first five rows of CSV dataset.

Step 5 :- Call tail method on DataFrame df to retrieve the last five rows of CSV dataset.

Step 6 :- Print first five lines and last five lines of dataset to standard terminal console display.

Step 7 :- Terminate execution of Pandas file processing script and clear DataFrame structures from system memory.

PROGRAM

```
import pandas as pd

filename = 'example.csv'
df = pd.read_csv(filename)

print("First five lines of the file:")
print(df.head())

print("\nLast five lines of the file:")
print(df.tail())
```

SAMPLE OUTPUT

Terminal Output

```
First five lines of the file:
Name Age City
0 Alice 30 New York
1 Bob 25 Los Angeles
2 Charlie 35 Chicago
3 David 40 Houston
4 Eve 28 Phoenix

Last five lines of the file:
Name Age City
5 Frank 33 Philadelphia
6 Grace 45 San Antonio
7 Hannah 27 San Diego
8 Ivan 50 Dallas
9 Judy 29 San Jose
```

RESULT

The Pandas CSV data reading and head/tail preview program was verified successfully.

EXPERIMENT NO: 16 **Date:** _____

MATPLOTLIB

AIM

Write a program to draw geometric shapes using matplotlib

OBJECTIVES

- To initialize Matplotlib figure and subplot axis objects.
- To render geometric patch primitives (Rectangle, Circle, Polygon).
- To format plot aesthetics including labels, title, grid, aspect ratio, and show display.

ALGORITHM

Step 1 :- Initialize graphic visualization program using Matplotlib module to render geometric shapes on plot canvas.

Step 2 :- Import matplotlib.pyplot module under alias plt and numpy module under alias np for drawing.

Step 3 :- Create figure and subplots axis objects using plt.subplots function to define plot drawing canvas.

Step 4 :- Create Rectangle patch specifying position, width, height, color, and alpha transparency, adding to axis.

Step 5 :- Create Circle patch specifying center position, radius, color, and transparency, adding to plot axis.

Step 6 :- Create Polygon patch with NumPy array triangle coordinates, adding red triangle patch to axis.

Step 7 :- Set x and y axis limits from zero to one and set equal aspect ratio scaling.

Step 8 :- Set grid, axis labels, title, and display rendered figure window using plt.show function.

Step 9 :- Close graphic visualization window and terminate program execution releasing plot resources back to system.

PROGRAM

```
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots()

rectangle = plt.Rectangle((0.2, 0.5), 0.4, 0.3, color='blue', alpha=0.3)
ax.add_patch(rectangle)

circle = plt.Circle((0.8, 0.2), 0.1, color='green', alpha=0.5)
ax.add_patch(circle)

```

```
triangle_points = np.array([[0.6, 0.8], [0.8, 0.8], [0.7, 0.9], [0.6, 0.8]])
triangle = plt.Polygon(triangle_points, closed=True, color='red', alpha=0.4)
ax.add_patch(triangle)

ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.set_aspect('equal', adjustable='box')

ax.grid(True)
ax.set_xlabel('X-axis')
ax.set_ylabel('Y-axis')
ax.set_title('Geometric Shapes')

plt.show()
```

SAMPLE OUTPUT

Terminal Output

```
[Plot Window Displayed:  Rendered Blue Rectangle, Green Circle, and Red Triangle
shapes on grid axes]
```

RESULT

The Matplotlib geometric shape rendering program was successfully executed and visualized.